Common Problems

Elevator

By Evan King · Updated Jan 19, 2026 · medium · Asked at:

Try This Problem Yourself
Practice with guided hints and real-time feedback

Start Practice

Watch Video Walkthrough
Watch the author walk through the problem step-by-step

Watch Now

Understanding the Problem

What is an Elevator System?

An elevator system manages multiple elevators serving different floors in a building. When someone requests an elevator, the system decides which one to dispatch. Once inside, passengers select their destination floors. The system needs to move elevators efficiently while handling multiple concurrent requests.

Requirements

As the interview begins, we'll likely be greeted by a simple prompt to set the stage for the system we need to design.

> "Design an elevator control system for a building. The system should handle multiple elevators, floor requests, and move elevators efficiently to service requests."

Before jumping into class design, we should ask questions of our interviewer.

Clarifying Questions

A reliable way to structure our questions is to cover four areas: what the core actions are, how errors should be handled, what the boundaries of the system are, and whether we need to plan for future extensions.

> The goal here is to turn that vague prompt into a concrete specification, something we can actually build against.

Here's how a conversation between you and the interviewer might go. In a real interview, this back-and-forth typically takes 1 to 3 minutes. Don't rush it. Getting clear requirements upfront saves time later.

> **You:** "How many elevators and floors are we dealing with? Is this configurable or fixed?"

> **Interviewer:** "Let's say 3 elevators serving 10 floors, numbered 0 through 9. Keep it fixed for now."

Good. We've established the scale. This matters because the complexity changes dramatically with scale. Three elevators is very different from 300.

> **You:** "When someone on a floor calls an elevator, is it a simple up/down hall call or a destination dispatch panel where they pick a floor and get assigned a car? Either way, how should the system decide which elevator to send?"
> **Interviewer:** "Keep it simple: classic up/down hall calls. The system should pick an elevator intelligently, but we can start with something straightforward."

This tells us we're modeling the usual two-button hall calls (not destination dispatch), and dispatch logic is in scope with flexibility in how sophisticated it needs to be.

> **You:** "Once someone is inside an elevator, how do they select floors? Can they select multiple destinations?"
> **Interviewer:** "Yes, they can press multiple floor buttons inside the elevator."

Now we know elevators need to handle multiple destination requests, not just one at a time.

> **You:** "Should we distinguish between different types of stops? Like someone on a floor calling the elevator up versus down, versus someone inside selecting a destination?"
> **Interviewer:** "Good thinking. Yes, hall calls should specify direction. Destination buttons inside don't have a direction - just stop there."

This is important. We now know there are two distinct types of stops: hall calls with a direction (UP or DOWN), and destination requests without a direction. This will influence our design later.

> **You:** "What about invalid requests? Should we handle things like requesting a floor that doesn't exist?"
> **Interviewer:** "Yes, reject those clearly. Return false, but don't let invalid input break the system. If someone requests the floor they're already on, just treat it as a no-op - they're already there."

> **You:** "Should we worry about elevator capacity, weight limits, door mechanics, or emergency stops?"
> **Interviewer:** "No, keep it focused on the scheduling and movement logic."



> This question saved us from spending 15 minutes designing door state machines and weight sensors. In interviews, knowing what to explicitly exclude is just as important as knowing what to include. You only have 35 minutes.

> **You:** "One last thing. Are we building a simulation where I control time with something like a `step()` function, or are we modeling the actual control software that would interface with hardware?"
> **Interviewer:** "Go with the simulation model. Discrete time steps work well. We can advance time one tick at a time."

That question about simulation vs. control software might seem overly specific, but it's one of the most important clarifications we can make in this type of interview.

A real elevator system doesn't have a `step()` function we call to advance time. It has motor controllers that physically move the car, floor sensors that fire interrupts on arrival, and asynchronous control flow driven by

hardware events. You'd see callbacks like `onFloorReached(floor)` instead of manually incrementing `currentFloor++`.

But in LLD interviews, we're almost always expected to build a simulation. Whether the problem involves elevators, parking lots, vending machines, or traffic lights, the pattern is the same. Abstract away the hardware and control time yourself with `step()` or `tick()`. This keeps the problem tractable in 35 minutes, makes the logic deterministic and testable, and lets us focus on object modeling and algorithms rather than concurrency and hardware interfaces.

> When you encounter a problem like this, ask explicitly, "Are we building a simulation where I control time, or modeling actual control software?" This signals you understand the distinction (a senior-level insight) and prevents you from going down the wrong path. Nine times out of ten, they want the simulation, and you can confidently focus on your object model and scheduling algorithm rather than motor controllers and sensor interrupts.

Final Requirements

After that back-and-forth, we can write down the final requirements, as well as any learning about what is out of scope, on the whiteboard.

FINAL REQUIREMENTS

```
Requirements:
1. System manages 3 elevators serving 10 floors (0-9)
2. Users can request an elevator from any floor (hall call). System decides which elevator to dispatch.
3. Once inside, users can select one or more destination floors
4. Simulation runs in discrete time steps (e.g., a `step()` or `tick()` call advances time)
5. Elevator stops come in two types:
    - Hall calls: Request from a floor with direction (UP or DOWN)
    - Destination: Request from inside elevator (no direction specified)
6. System handles multiple concurrent pickup requests across floors
7. Invalid requests should be rejected (return false)
    - Non-existent floor numbers
8. Requests for the current floor are treated as a no-op / already served (doors out of scope)

Out of scope:
- Weight capacity and passenger limits
- Door open/close mechanics
- Emergency stop functionality
- Dynamic floor/elevator configuration
- UI/rendering layer
```

Core Entities and Relationships

Now that we've locked down the requirements, we need to identify the core objects that will make up our system.

The trick is to scan through our requirements and spot the key entities. These are usually nouns that seem to have behavior or state. For an elevator system, a few obvious candidates jump out: elevators, floors, stops/requests, and some kind of central coordinator that dispatches work.

Let's think through each potential entity before narrowing it down to the final list.

Floor - At first glance, floors seem important. But what would a Floor class actually do? In our requirements, floors don't maintain state or enforce rules. They're just numbers that identify positions in the building. A floor doesn't "do" anything. This stays as an integer, not a class.

Request - When someone presses a button (either a hall call or destination), should we model this as a class? A Request would contain the floor number and potentially the type (hall call with direction vs destination). But is this just wrapping primitives, or does it have real behavior? This is an interesting question. We'll defer this decision for now and revisit it during class design. We'll see whether we actually need a Request class or if storing just floor numbers is sufficient.

Elevator - Elevators definitely maintain state (current floor, direction, which floors to stop at) and enforce rules (must service stops along their path, can't go below floor 0). Clear candidate for an entity.

ElevatorController - Someone needs to receive hall calls and decide which elevator to dispatch. This is the orchestrator. It owns the system-level view of all elevators and makes coordination decisions. Another clear entity.



> Whenever we're building a tick-based simulation, we'll need a controller entity that owns the `step()` function. This controller advances time for the entire system and orchestrates all the actors.

For this system, we land on three core entities:

Entity	Responsibility
ElevatorController	The orchestrator. Receives hall calls from people on floors, decides which elevator should handle each request, and coordinates the overall system. Doesn't need to know the internals of how elevators move. It just dispatches requests and tells elevators to advance.
Elevator	Represents one elevator in the building. Maintains its current floor, direction, and queue of requests. Knows how to execute the movement behavior. Move one floor at a time, stop when needed, reverse when there are no more stops ahead. Doesn't know about other elevators.
Request	Represents a stop the elevator needs to make. We're not sure yet if this needs to be its own class or just a floor number - we'll decide during class design once we understand the movement logic better.



> It's reasonable to be on the fence about whether something should be a full entity or just a primitive. Communicate this to your interviewer and come back to it as you learn more during class design.

Class Design

With our core entities locked in, it's time to move on to flesh out what each one actually looks like. What state it holds and what operations it supports.

The smart move here is to work from the top down. Start with the `ElevatorController` since it's the entry point. External code interacts with the controller, not individual elevators. Design its interface first, then drill into the `Elevator` class. This approach keeps us thinking about the public contract instead of getting bogged down in implementation.

For both classes, we'll trace back to the requirements and ask what does this entity need to remember, and what actions does it need to support? Let's tackle ElevatorController first.

ElevatorController

We can derive almost everything about `ElevatorController` directly from the final set of requirements. During the interview, revisit the requirements and ask "What does the controller need to remember to enforce this?" This is how we'll derive the state.

Requirement	What `ElevatorController` must track
"System manages 3 elevators serving 10 floors"	The collection of elevators it controls

This leads to the following state:

ELEVATORCONTROLLER STATE

```
class ElevatorController:
    - elevators: List<Elevator>
```

In our design, we'll choose to immediately dispatch hall calls to an elevator when they arrive so the controller only needs the list of elevators and doesn't maintain a queue of unassigned requests. It picks an elevator right away and tells that elevator to add the request to its queue. This keeps the controller stateless beyond just holding the elevators.

> We could design this differently. Maintain a queue of pending requests on the controller and have elevators pull from it. Both approaches work. The immediate dispatch model is simpler for an interview, but if the interviewer asks "what if all elevators are busy?", you'd want the queue model. Always be ready to explain your tradeoffs.

Next, look at the actions the outside world needs to perform. Again, every method on `ElevatorController` should correspond to a concrete need in the problem statement.

Need from requirements	Method on `ElevatorController`
"Users can request an elevator from any floor"	`requestElevator(floor, type)` for the hall call entry point
"Discrete time steps"	`step()` to advance all elevators one tick

Adding the methods to the `ElevatorController` class, we get:

ELEVATORCONTROLLER

```
class ElevatorController:
    - elevators: List<Elevator>

    + ElevatorController()
    + requestElevator(floor, type) -> boolean
    + step() -> void
```

Note that `requestElevator` takes a `RequestType` (PICKUP_UP or PICKUP_DOWN), not a `Direction`. We don't reuse the `Direction` enum here because hall calls are never IDLE — accepting `Direction` would let invalid values in and require runtime validation to catch them. Using `RequestType` makes that impossible at the type level.

The constructor initializes the three elevators:

ELEVATORCONTROLLER CONSTRUCTOR

```
ElevatorController()
    elevators = [
        Elevator(),
        Elevator(),
        Elevator()
    ]
```

`requestElevator` is where dispatch logic lives. When someone on floor 5 presses the "up" button, this method decides which of the 3 elevators should respond. We'll implement this later, but the signature captures the intent. Give me a floor and a request type, and I'll dispatch an elevator.

`step` is how time advances in our simulation. Each call represents one unit of time passing. The controller tells each elevator to take one step. Move one floor, handle stops, or update direction.

Elevator

`Elevator` represents one elevator in the building. From the requirements, we need to track position, movement direction, and which floors to visit.

We can derive its state directly from the problem:

Requirement	What `Elevator` must track
"Elevators serving 10 floors (0-9)"	Current floor position
"Continue in current direction servicing all requests"	Current direction of travel
"Once inside, users can select one or more destination floors"	Collection of floors to stop at
"System manages 3 elevators"	No extra state needed beyond maintaining multiple instances

This gives us the following state:

ELEVATOR STATE

```
class Elevator:
    - currentFloor: int
    - direction: Direction        // UP, DOWN, IDLE
    - requests: Set<???>
```

But what type should `requests` be? This is the question we deferred during entities. Should we store just floor numbers, or something richer? Let's think through the options.

Good Solution: Store Just Floor Numbers (Set<Integer>)

Approach

The simplest option is to store which floors we need to visit, without tracking why.

SIMPLE APPROACH

```
class Elevator:
    - requests: Set<Integer>
```

```
addRequest(floor)
    requests.add(floor)

step()
    if requests.contains(currentFloor)
        requests.remove(currentFloor)
        // ... continue moving
```

This works. If multiple people want floor 7 for different reasons (going up, going down, or as a destination), we just stop there once.

Challenges

But there is a problem with this approach. Imagine an elevator at floor 5 going UP with requests for `[7, 8]`.

Someone on floor 7 presses the DOWN button. They want to go down. We add floor 7 to requests, which is already there, so nothing changes.

When the elevator arrives at floor 7 going UP, it stops. The person waiting sees the elevator going UP and gets on, even though they wanted DOWN. Now they're trapped riding up to floor 8, then back down to 7.

This isn't just inefficient. It's confusing. Real passengers see the direction indicator on the elevator. If they're waiting to go down and an UP elevator stops, it creates an awkward choice: get on and ride the wrong way, or wait and hope it comes back?

For a junior level interview, this simplification might be acceptable. But we can do better.

Great Solution: Request Class with Type

Approach

Instead of just storing floor numbers, create a `Request` class that captures floor + request type (pickup up/down or destination).

REQUEST CLASS

```
enum RequestType:
    PICKUP_UP      // Hall call going up
    PICKUP_DOWN    // Hall call going down
    DESTINATION    // Destination button (stop regardless of elevator direction)

class Request:
    - floor: int
    - type: RequestType

    + Request(floor, type)
```

```
// equals() and hashCode() based on floor + type
// So Request(7, PICKUP_UP) != Request(7, PICKUP_DOWN) != Request(7, DESTINATION)
```

Now our Elevator uses a Set of Requests instead of integers:

DIRECTION-AWARE APPROACH

```
class Elevator:
    - requests: Set<Request>
```

```
addRequest(floor, type)
    requests.add(Request(floor, type))

step()
    // Check if we should stop based on floor and type
    pickupType = (direction == UP) ? PICKUP_UP : PICKUP_DOWN
    pickupRequest = Request(currentFloor, pickupType)
    destinationRequest = Request(currentFloor, DESTINATION)

    if requests.contains(pickupRequest) || requests.contains(destinationRequest)
        requests.remove(pickupRequest)
        requests.remove(destinationRequest)
        // ... continue moving
```

We can consider the same scenario but with these changes: elevator at floor 5 going UP, person on floor 7 wants DOWN.

We add `Request(7, PICKUP_DOWN)` to requests. When the elevator reaches floor 7 going UP, it checks for `Request(7, PICKUP_UP)` or `Request(7, DESTINATION)`. Neither exists, so it doesn't stop. It continues to floor 8, reverses, and comes back DOWN. Now when it reaches floor 7 going DOWN, it finds `Request(7, PICKUP_DOWN)` and stops.

The person sees the elevator going the direction they want. No confusion, no awkward ride in the wrong direction.

The Request class gives us direction-aware stopping, which is critical for proper elevator behavior. The tradeoff is slightly more complexity (a new class, equals/hashCode implementation), but the benefits are worth it here.

Great, so we can land on `Request` being a class with a type so our elevator state looks like this:

ELEVATOR STATE

```
class Elevator:
    - currentFloor: int
    - direction: Direction        // UP, DOWN, IDLE
    - requests: Set<Request>
```

Why does `direction` need an IDLE state? When an elevator has no requests, it's not moving up or down. It's idle. We need an explicit state to represent "not moving" so the elevator doesn't keep drifting up or down forever. Some candidates try to get away with just UP and DOWN, then realize they can't handle the "no requests" case cleanly.

With the state decisions made, let's define what methods `Elevator` needs to support from the outside:

Need from requirements	Method on `Elevator`
"Users can select one or more destination floors"	`addRequest(request)` to add stop to queue
"Discrete time steps, elevator moves"	`step()` to execute one tick of movement
Controller needs to know where elevator is	`getCurrentFloor()` to return current position
Controller needs to know direction for dispatch	`getDirection()` to return current direction

That gives us this interface:

ELEVATOR

```
class Elevator:
    - currentFloor: int
    - direction: Direction        // UP, DOWN, IDLE
    - requests: Set<Request>      // Request = (floor, RequestType)

    + Elevator()
    + addRequest(request) -> boolean
```

```
    + step() -> void
    + getCurrentFloor() -> int
    + getDirection() -> Direction
```

The constructor initializes all elevators at the ground floor:

ELEVATOR CONSTRUCTOR

```
Elevator()
    currentFloor = 0  // all elevators start at ground floor
    direction = IDLE
    requests = HashSet()
```

`addRequest` is a unified interface used both by the controller (for hall calls) and by passengers (for destinations). The elevator doesn't care why it's stopping at floor 5. It just adds the request to the queue.

`step` is where the movement logic lives. This is the heart of the elevator. The method that decides whether to move, stop, reverse, or go idle.

> You might notice that `ElevatorController.requestElevator(floor, type)` and `Elevator.addRequest(request)` both seem to "add a request" and be tempted to extract a common interface like `IRequestHandler`. This is a trap. These methods are doing fundamentally different things: `requestElevator` is a coordination operation that picks which elevator should handle a request, while `addRequest` is a simple state mutation on a single elevator. Creating a shared interface would be forcing an abstraction that doesn't reflect any real polymorphic behavior. The controller is not a type of elevator, and they're not interchangeable. Just because two methods happen to accept similar parameters doesn't mean they should share an interface. Save interfaces for when you actually need polymorphism - when you have multiple implementations of the same behavior that need to be substitutable.

Request

Since we decided we needed a Request class when designing `Elevator`, let's formalize its interface.

REQUEST

```
class Request:
    - floor: int
    - type: RequestType

    + Request(floor, type)
    + getFloor() -> int
    + getType() -> RequestType

enum RequestType:
    PICKUP_UP
    PICKUP_DOWN
    DESTINATION
```

That's it! Nice and simple, but this will allow us to have a more intelligent implementation of the elevator algorithm in the next section.

Final Class Design

We end up with the following class design:

FINAL CLASS DESIGN

```
class ElevatorController:
    - elevators: List<Elevator>

    + ElevatorController()
    + requestElevator(floor, type) -> boolean
    + step() -> void

class Elevator:
    - currentFloor: int
    - direction: Direction        // UP, DOWN, IDLE
    - requests: Set<Request>

    + Elevator()
    + addRequest(request) -> boolean
    + step() -> void
    + getCurrentFloor() -> int
    + getDirection() -> Direction

class Request:
    - floor: int
    - type: RequestType

    + Request(floor, type)
    + getFloor() -> int
    + getType() -> RequestType

enum Direction:
    UP
    DOWN
    IDLE

enum RequestType:
    PICKUP_UP
    PICKUP_DOWN
    DESTINATION
```

The design encapsulates movement logic within Elevator while ElevatorController handles system-wide coordination. Request as a separate class enables direction-aware stopping behavior without coupling the algorithm to external details.

Implementation

Now that we've defined our classes, it's time to implement the actual method bodies. Before diving in, check with your interviewer - some want working code, others prefer pseudocode, and some just want you to talk through the logic. Adjust your level of detail based on what they're looking for. We'll stick to pseudocode, which is the most common, but will include complete implementations in common languages at the bottom of this section.

When implementing each method, we'll use this approach:

1. **Start with the main flow** - What happens in the normal case when everything goes right?
2. **Handle the edge cases** - What about invalid inputs, boundary conditions, or unexpected states?

We'll follow the same top-down approach from Class Design. The most interesting methods are `ElevatorController.requestElevator()` (dispatch logic) and `Elevator.step()` (the movement logic).

Aim for clarity over cleverness. Don't make things more complicated than they need to be.

ElevatorController

Let's implement `requestElevator` first.

Core logic

1. Validate the floor number
2. Pick which elevator should handle this request
3. Tell that elevator to add the floor to its stops

Edge cases

- Floor out of bounds (less than 0 or greater than 9)
- Invalid direction

Here's a basic implementation in pseudocode:

REQUESTELEVATOR()

```
requestElevator(floor, type)
    // Validate
    if floor < 0 || floor > 9
        return false
    if type == DESTINATION
        return false  // hall calls only

    // Create Request at the boundary, pass it through the system
    request = Request(floor, type)
    best = selectBestElevator(request)
    return best.addRequest(request)
```

The interesting part is `selectBestElevator`. There are multiple strategies here, and this is where we can have a good discussion with our interviewer about tradeoffs. Start with the simplest approach. After implementing it, proactively mention to your interviewer: "This works, but I could make it more sophisticated by considering direction. Would you like me to implement that?" This shows you're thinking ahead without over-engineering upfront.

Bad Solution: Nearest Elevator (Ignore Direction)

Approach

The simplest strategy is to pick the elevator closest to the requested floor, regardless of which direction it's heading.

NEAREST ELEVATOR STRATEGY

```
selectBestElevator(request)
    floor = request.getFloor()
    nearest = elevators[0]
    minDistance = abs(elevators[0].getCurrentFloor() - floor)

    for e in elevators
        distance = abs(e.getCurrentFloor() - floor)
        if distance < minDistance
            minDistance = distance
            nearest = e

    return nearest
```

This is correct and takes about 30 seconds to implement. For a small building with light traffic, this works fine.

Challenges

The problem becomes obvious with a simple example. Someone on floor 5 presses "up". The nearest elevator is on floor 6, but it's going down to floor 1. We'd send that elevator, even though it's heading the wrong way.

Thanks to our `Request` class with direction types, the elevator won't actually pick them up while going down - it'll pass by floor 5, go to floor 1, reverse, and come back up. But that's a long wait. The person on floor 5 watches the nearest elevator ignore them as it goes the wrong way. Poor experience.

Good Solution: Direction-Aware (Basic)

Great Solution: Direction-Aware with Request Queue Analysis

> We designed this for a single elevator scheduling algorithm. If you ran an elevator company serving different buildings with different needs — some prioritize wait time, others prioritize energy efficiency — you'd swap in different scheduling strategies. Each strategy implements the same `selectBestElevator(request)` interface but picks elevators differently. That's the Strategy pattern in action.

The last method is `step`:

STEP()

```
step()
    for e in elevators
        e.step()
```

That's it. Just tell each elevator to advance one tick. The controller doesn't need to know how elevators move. That's encapsulated in `Elevator.step()`.

Elevator

Now that we've implemented the controller, let's drill into the elevator movement logic. The heart of the elevator system is `Elevator.step()`.

During requirements, the interviewer left the movement behavior up to us. We need to decide how elevators should handle multiple stops. Before jumping to code, let's explore different movement algorithms and understand the tradeoffs.

> This is pretty implementation heavy. Depending on your interview, you may stay higher level and just describe the different algorithms and their tradeoffs. We'll go all the way through the implementation for completeness.

Bad Solution: FIFO (First-In-First-Out)

Approach

The simplest movement strategy is to service requests in the order they were received, regardless of where they are or which direction we're heading. If we modeled requests as a queue instead of a set:

FIFO MOVEMENT (USING QUEUE<REQUEST>)

```
step()
    if requestQueue.isEmpty()
        direction = IDLE
        return

    // Peek at the oldest request (don't remove yet)
    target = requestQueue.peek()

    // Move toward it
    if currentFloor < target.getFloor()
        currentFloor++
    else if currentFloor > target.getFloor()
        currentFloor--

    // Remove request when we arrive
    if currentFloor == target.getFloor()
        requestQueue.poll()
```

For example, if the elevator is at floor 5 with the queue `[Request(8), Request(3), Request(7)]` (in arrival order):

- Go to 8 first (5→6→7→8)
- Then immediately go to 3 (8→7→6→5→4→3)
- Then go to 7 (3→4→5→6→7)

Total movement: 3 floors up + 5 floors down + 4 floors up = 12 floors traveled.

Challenges

This creates constant direction changes and poor passenger experience. The person who requested floor 7 watches the elevator go to 8, then all the way down to 3, then finally come back up to 7. It feels random and inefficient. In a busy building, this would create extremely long wait times and passengers would feel like the elevator is ignoring them as it bounces back and forth.

Good Solution: Always Go to Nearest Stop

Approach

Better than FIFO is to always head to whichever stop is closest to our current position, regardless of direction.

NEAREST-FIRST MOVEMENT

```
step()
    if requests.isEmpty()
        direction = IDLE
```

```
        return

    // Find nearest request (with lowest floor as tiebreaker for determinism)
    nearest = null
    minDistance = Integer.MAX_VALUE

    for request in requests
        distance = abs(currentFloor - request.getFloor())
        if distance < minDistance ||
            (distance == minDistance && (nearest == null || request.getFloor() < nearest.getFloor()))
            minDistance = distance
            nearest = request

    // Move toward it
    if currentFloor < nearest.getFloor()
        currentFloor++
    else if currentFloor > nearest.getFloor()
        currentFloor--

    // Stop if we arrived
    if currentFloor == nearest.getFloor()
        requests.remove(nearest)
```

Same scenario: elevator at floor 5 with requests `[Request(8, DESTINATION), Request(3, PICKUP_DOWN), Request(7, PICKUP_UP)]`:

- Nearest is floor 3 (2 floors away), so go there first (5→4→3)
- Then nearest is floor 7 (4 floors away), go there (3→4→5→6→7)
- Then go to floor 8 (7→8)

Total movement: 2 floors down + 4 floors up + 1 floor up = 7 floors traveled.

This is better than FIFO! But there's still a problem.

Challenges

We're still changing direction unnecessarily. When at floor 5 with stops at floors 8, 3, and 7, we had stops both above (7, 8) and below (3). The nearest-first approach sent us down to 3 first, then back up to 7 and 8. But we were already positioned perfectly to sweep up through 7 and 8, then come back down to 3. The algorithm doesn't preserve direction, so it still creates awkward movement patterns. A passenger at floor 7 watching the elevator at floor 5 would see it go down to 3 first before coming to get them, even though 7 is right above.

Great Solution: SCAN (Continue Until Clear, Then Reverse)

Approach

The optimal strategy is to continue in your current direction, servicing all stops along the way, and only reverse when there are no more stops ahead. This is the SCAN algorithm.

SCAN MOVEMENT (WHAT WE'LL IMPLEMENT)

```
step()
    if requests.isEmpty()
        direction = IDLE
        return

    // Pick direction if idle based on nearest request
    if direction == IDLE
        direction = directionTowardNearestRequest()

    // Check if we should stop at current floor (direction-aware)
    if shouldStopHere()
        removeCurrentFloorRequests()
        if requests.isEmpty()
            direction = IDLE
            return

    // Reverse if no requests ahead in current direction
    if noRequestsAhead()
        reverseDirection()

    // Move one floor
    moveOneFloor()
```

Same scenario: elevator at floor 5 going UP with requests at floors 3, 7, and 8:

- Continue UP through floor 7 (5→6→7, stop)
- Continue UP to floor 8 (7→8, stop)
- No more requests above, reverse to DOWN
- Go to floor 3 (8→7→6→5→4→3, stop)

Total movement: 3 floors up + 5 floors down = 8 floors traveled.

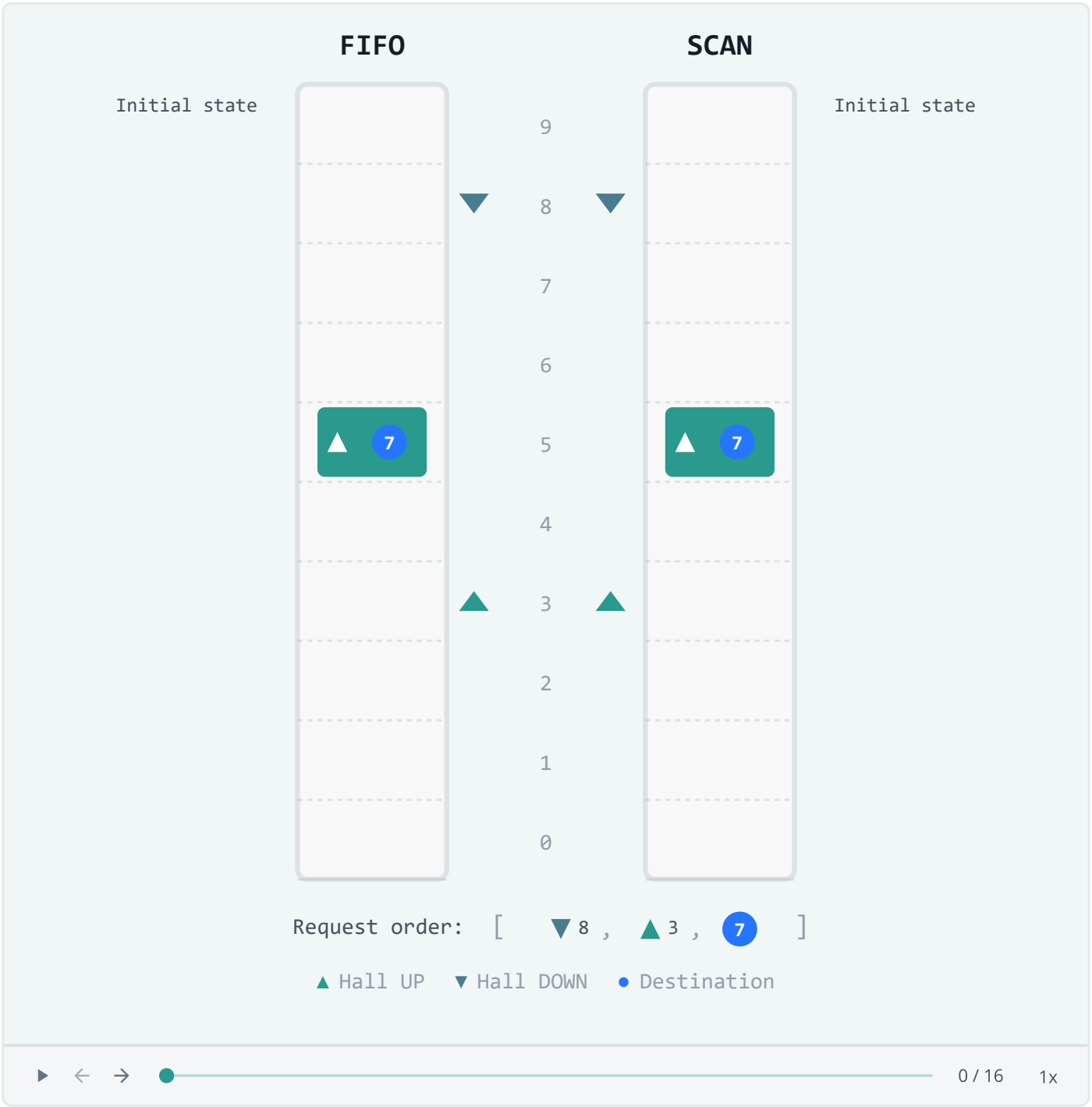
Wait, that's one more floor than nearest-first! Why is this better?

The key is not about absolute distance traveled, but about minimizing direction changes and passenger wait time. FIFO causes two reversals (up, then down, then back up), while both nearest-first and SCAN only reverse once. But SCAN has a crucial advantage over nearest-first: predictable behavior that matches passenger intuition.

With SCAN, the person at floor 7 sees the elevator coming toward them and it actually arrives. With nearest-first, they watch the elevator go the wrong way first, heading down to floor 3 before coming back. This creates confusion and frustration. Real passengers care about whether the elevator appears to be coming toward them or not.

In a busy building with constant requests, SCAN prevents the "thrashing" problem where the elevator bounces back and forth between nearby floors. It sweeps through all requests in one direction, which naturally batches nearby pickups and creates smooth, efficient movement. This is the same principle used in real elevator control systems and in disk scheduling algorithms for operating systems. The predictability and minimal direction changes make it optimal for concurrent request patterns.

The visualization below shows the difference between FIFO and SCAN on the same scenario. Step through to see how FIFO bounces back and forth inefficiently, while SCAN sweeps smoothly in one direction before reversing.



Same requests, different algorithms. FIFO bounces 8→3→7. SCAN sweeps 7→8→3.

We'll choose the SCAN algorithm from our set of options, but you and your interviewer may agree to go with a simpler algorithm.

When it comes to implementation, the high-level logic is simple, but the devil is in handling edge cases correctly.

Edge cases to handle

- Empty requests - Should go IDLE and do nothing

- Stopped at a floor - Need to remove the request, check if we should reverse or go idle
- IDLE with requests - Need to pick a direction before doing anything else
- No requests ahead in current direction - Need to reverse

Here's the full implementation:

STEP()

```
step()
    // Case 1: Nothing to do
    if requests.isEmpty()
        direction = IDLE
        return

    // Case 2: If idle, pick a direction based on nearest request
    if direction == IDLE
        // Find the nearest request to establish initial direction
        nearest = null
        minDistance = Integer.MAX_VALUE

        for req in requests
            distance = abs(req.getFloor() - currentFloor)
            if distance < minDistance ||
                (distance == minDistance && (nearest == null || req.getFloor() < nearest.getFloor()))
                minDistance = distance
                nearest = req

        direction = (nearest.getFloor() > currentFloor) ? UP : DOWN

    // Case 3: Check if we should stop at current floor
    // Check pickup requests matching our direction, plus any destination requests
    pickupType = (direction == UP) ? PICKUP_UP : PICKUP_DOWN
    pickupRequest = Request(currentFloor, pickupType)
    destinationRequest = Request(currentFloor, DESTINATION)

    if requests.contains(pickupRequest) || requests.contains(destinationRequest)
        requests.remove(pickupRequest)
        requests.remove(destinationRequest)
        // Note: If Request(currentFloor, PICKUP_DOWN) exists but we're going UP,
        // it survives and will be serviced on the return trip going DOWN.
        // This is correct - we only pick up passengers going our direction.

        if requests.isEmpty()
            direction = IDLE
        return  // we stopped this tick, don't move

    // Case 4: Reverse if no requests ahead
    if !hasRequestsAhead(direction)
        direction = (direction == UP) ? DOWN : UP
        return  // don't move this tick, let next tick check for stops

    // Case 5: Move one floor
    if direction == UP
        currentFloor++
    else if direction == DOWN
        currentFloor--
```

Let's walk through why each case matters:

Case 1: Nothing to do. If requests is empty, set direction to IDLE and return. Without this, an idle elevator with no requests would fall through to the movement logic and start drifting.

Case 2: Pick direction if idle. If direction is IDLE but requests isn't empty, we need to pick a direction before checking for stops. We find the nearest request to establish direction. If there's a tie (multiple requests at the same distance), we pick the one with the lower floor number as a tiebreaker. This establishes an initial direction so the stop-check logic knows which pickup type to look for.



> Don't use `requests.iterator().next()` here! Since `requests` is a HashSet, iterator order is non-deterministic. Your simulation would behave differently across runs, making it impossible to debug or verify behavior. Always use a deterministic rule when picking from a set: nearest request, lowest floor, or some other consistent tiebreaker.

Case 3: Stopping at current floor. Now that we have a direction, check if we should stop. When we remove a request, we check whether we have more requests. If not, go IDLE. Either way, return — we don't move on the same tick we stop. Some candidates forget the return and move the elevator after stopping, which creates weird behavior. If we need to reverse (because no requests are ahead), Case 4 handles that on the next tick.

Case 4: No requests ahead. This is the reversal logic. If we're going UP but all remaining requests are below us, reverse. If we're going DOWN but all remaining requests are above us, reverse. We return here without moving so the next tick can check if there's a stop at the current floor in the new direction.

Case 5: Move. Finally, move one floor in the current direction.

You'll notice we've been using `hasRequestsAhead(direction)` in Case 4. This helper answers a simple question: "Are there any requests above me (if going UP) or below me (if going DOWN)?" If yes, keep going. If no, reverse.

HASREQUESTSAHEAD()

```
hasRequestsAhead(dir)
    for request in requests
        if dir == UP && request.getFloor() > currentFloor
            return true
        if dir == DOWN && request.getFloor() < currentFloor
            return true
    return false
```

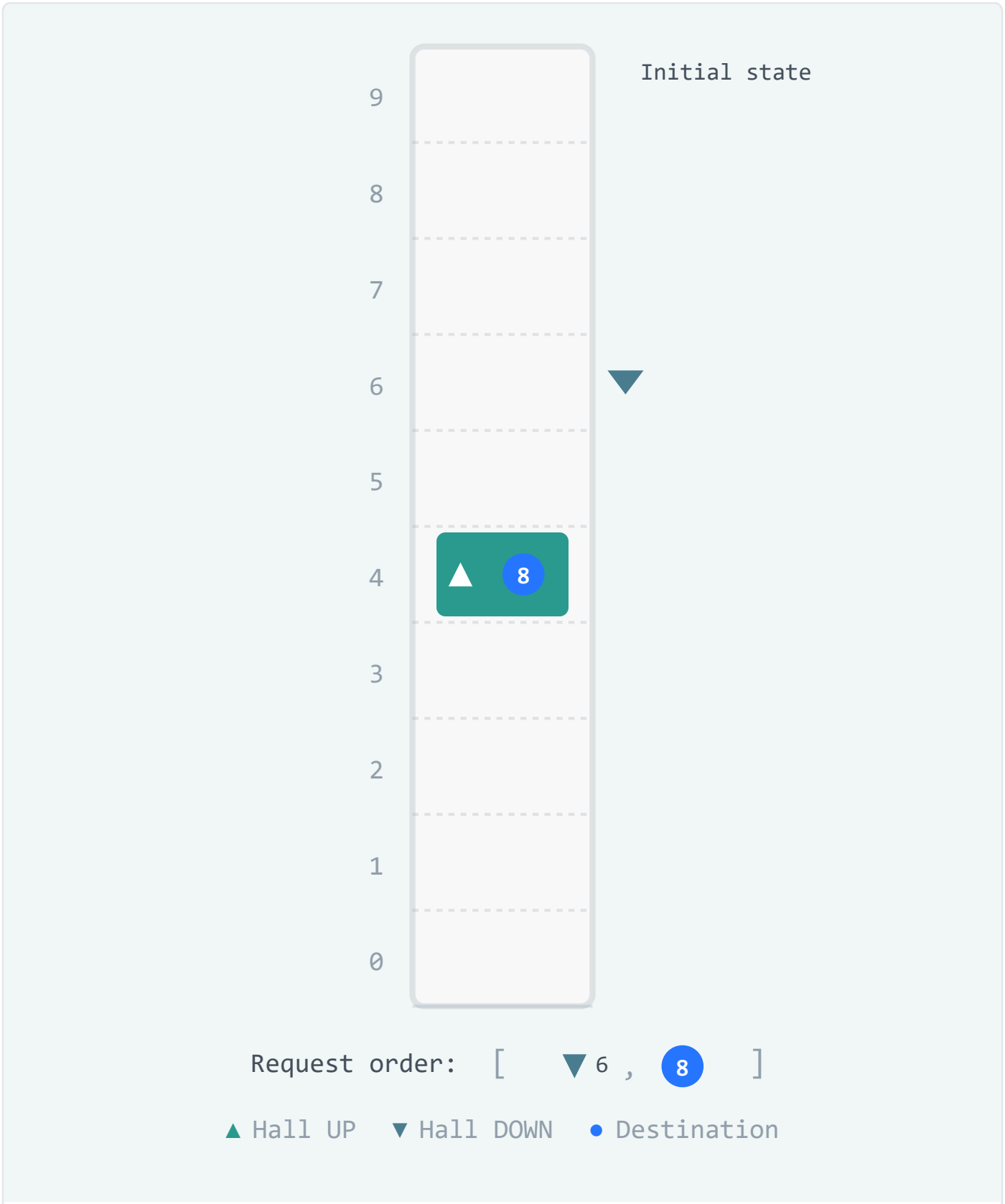
Notice this checks for ANY request in that direction, regardless of type. This is intentional.

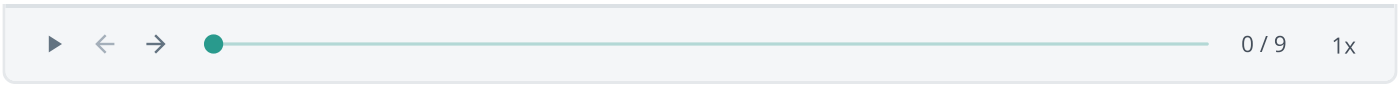


> Why does `hasRequestsAhead()` check for ANY request regardless of type? Because the elevator needs to *travel* toward all requests, but only *stop* for matching ones.
>
> - **Stopping** is direction-aware: only stop for passengers going your direction. When going UP, stop for `PICKUP_UP` and `DESTINATION` requests, not `PICKUP_DOWN`.

- **Traveling** is not: go toward ANY request, even if you won't stop for it until you reverse.

To make this click, imagine an elevator at floor 4 going UP with two requests: someone on floor 6 wants to go DOWN, and there's a destination button pressed for floor 8.





Watch: the elevator PASSES floor 6 going UP, stops at floor 8, reverses, then picks up floor 6 going DOWN.

The elevator *passes* floor 6 on the way up — the person waiting there sees it go by without stopping. But this isn't a bug! They pressed DOWN, and the elevator is going UP. Once the elevator reaches floor 8 (the destination), reverses to DOWN, and returns to floor 6, it finally stops. Now the passenger boards an elevator going their direction.

Notice we don't explicitly check for floors 0 and 9. Why not? Because `hasRequestsAhead` already handles it. If we're on floor 9 going UP, `hasRequestsAhead(UP)` returns false (no requests above floor 9 exist), so Case 4 reverses us to DOWN. If we're on floor 0 going DOWN, `hasRequestsAhead(DOWN)` returns false, so we reverse to UP. The boundary handling falls out naturally from the movement logic without needing special cases.

> A common mistake is when candidates add explicit checks like "if currentFloor == 9 then direction = DOWN". This creates a bug. Imagine we're on floor 9 going up, and someone adds floor 7 to stops. We'd reverse to DOWN correctly. But if we have the explicit check, we'd force DOWN even when there might be a stop at floor 9 itself. Let the movement logic handle direction changes. Don't hardcode boundaries.

The last piece is `addRequest`:

ADDREQUEST()

```
addRequest(request)
    if request.getFloor() < 0 || request.getFloor() > 9
        return false
    if request.getFloor() == currentFloor
        return true  // already here; treat as no-op
    return requests.add(request)
```

Simple. Validate the floor number, return true if we're already there, otherwise add the request to the set. The Set handles deduplication automatically based on Request's equals() method (which compares both floor and type).

Verification

Let's trace through a specific scenario to verify the movement algorithm works correctly. Walking through concrete examples like this helps catch logic bugs before your interviewer finds them.

Elevator is on floor 3, going UP, with requests `Request(5, PICKUP_UP)` and `Request(7, DESTINATION)`.

TICK-BY-TICK EXECUTION

```
Tick 0: currentFloor=3, direction=UP, requests={Request(5, PICKUP_UP), Request(7, DESTINATION)}
  - Not at a stop, move up
Tick 1: currentFloor=4, direction=UP, requests={Request(5, PICKUP_UP), Request(7, DESTINATION)}
  - Not at a stop, move up
Tick 2: currentFloor=5, direction=UP, requests={Request(5, PICKUP_UP), Request(7, DESTINATION)}
  - At floor 5! Check for Request(5, PICKUP_UP) - found! Remove it
```

```
  - requests={Request(7, DESTINATION)}, still have requests ahead, stay UP
  - Don't move this tick (we just stopped)
Tick 3: currentFloor=5, direction=UP, requests={Request(7, DESTINATION)}
  - Not at a stop, move up
Tick 4: currentFloor=6, direction=UP, requests={Request(7, DESTINATION)}
  - Not at a stop, move up
Tick 5: currentFloor=7, direction=UP, requests={Request(7, DESTINATION)}
  - At floor 7! Check for Request(7, DESTINATION) - found! Remove it
  - requests={}, no more requests, go IDLE
  - Don't move this tick
Tick 6: currentFloor=7, direction=IDLE, requests={}
  - No requests, stay idle
```

Now someone on floor 2 presses the call button going DOWN. The controller calls `requestElevator(2, DOWN)`, which dispatches to our elevator, adding `Request(2, PICKUP_DOWN)`.

TICK-BY-TICK EXECUTION

```
Tick 7: currentFloor=7, direction=IDLE, requests={Request(2, PICKUP_DOWN)}
  - requests not empty, pick direction
  - nearest request is floor 2, which is < 7, so direction=DOWN
  - hasRequestsAhead(DOWN)? Yes, Request(2, PICKUP_DOWN) is below us
  - Move down
Tick 8: currentFloor=6, direction=DOWN, requests={Request(2, PICKUP_DOWN)}
  - Not at a stop, move down
Tick 9: currentFloor=5, direction=DOWN, requests={Request(2, PICKUP_DOWN)}
  - Not at a stop, move down
...continues until reaching floor 2...
```

Notice how the elevator doesn't start moving until tick 7, after it receives the new request. The IDLE state is crucial.

Complete Code Implementation

While most companies only require pseudocode during interviews, some do ask for full implementations of at least a subset of the classes or methods. Below is a complete working implementation in common languages for reference.

Elevator.py | ElevatorController.py | Request.py | Python

```
            if request.get_type() in (RequestType.PICKUP_UP, RequestType.DESTINATION):
                return True
        if dir == Direction.DOWN and request.get_floor() <= floor:
            if request.get_type() in (RequestType.PICKUP_DOWN, RequestType.DESTINATION):
                return True
    return False

def get_current_floor(self):
    return self.current_floor

def get_direction(self):
    return self.direction
```

Extensibility

If time allows, interviewers will sometimes add small twists to test whether our design can evolve cleanly. You typically won't need to fully implement these changes. Just explain how the classes would adapt. The depth and quantity of the extensibility follow-ups correlate with the candidate's target level (junior, mid-level, senior). Junior candidates often won't get any, mid-level may get one or two, and senior candidates may be asked to go into more depth.

> If you're a junior engineer, feel free to skip this section and stop reading here! Only carry on if you're curious about the more advanced concepts.

Below are the most common ones for elevator systems, with more detail than you'd need in an actual interview.

1. "How would you add priority floors or an express elevator?"

Express elevators skip certain floors and only stop at major ones (like 0, 5, 9). Or maybe some VIP passengers get priority.

> "There are a few ways to do this. The simplest is to add a `priority` field to the `Request` class and use a priority queue. But this breaks our sweep-in-one-direction behavior - we'd be jumping around instead of moving efficiently.
> A better approach is to keep the standard movement logic for normal elevators and add a separate `isExpress` flag. In the controller's dispatch logic, if the request is for a priority floor, send the express elevator. That elevator would have a restricted `addRequest` method that only accepts floors 0, 5, and 9. The `Request` class doesn't need to change at all - it still stores floor and type, but the elevator validates which floors it accepts."

A response like this shows you're thinking about multiple options and can weigh between them, the key to senior+ design.

KEY CHANGES

```
class ElevatorController:
    - elevators: List<Elevator>
```

```
    - expressElevator: Elevator  // NEW: track which elevator is express

class Elevator:
    - isExpress: bool
    - expressFloors: Set<int> = {0, 5, 9}
```

```
addRequest(request)
    if request.getFloor() < 0 || request.getFloor() > 9
        return false
    if request.getFloor() == currentFloor
        return true  // already here; treat as no-op

    // NEW: Reject non-express floors if this is an express elevator
    if isExpress && !expressFloors.contains(request.getFloor())
        return false

    return requests.add(request)

// In ElevatorController dispatch logic:
selectBestElevator(request)
    // NEW: For express floors, prefer the express elevator when it's idle; otherwise fall through to normal se
    if request.getFloor() in {0, 5, 9} && expressElevator.getDirection() == IDLE
        return expressElevator
    // ... normal selection logic for regular elevators
```

2. "How would you add undo to cancel a floor request?"

Maybe someone accidentally pressed the wrong floor button. Can they cancel it?

> Fun fact: real elevators don't let you "undo" because a lit button signals to others that a stop is queued. Clearing it would silently drop requests from people who just trusted the light and didn't press again.

> "Since all requests flow through `addRequest`, I'd add a matching `removeRequest(request)` method to Elevator. It just removes the Request from the requests set. If someone cancels their request, call `removeRequest` with the same request they originally submitted, and the elevator won't stop there anymore.
> One edge case is what if they cancel while the elevator is already moving there? If they remove the request before the elevator arrives, it works fine. The elevator will skip it. If they try to remove it after the elevator has already stopped, `removeRequest` would just do nothing since the request is already gone. Either way, the core movement logic in `step()` doesn't change."

This works because our design already isolated state changes to a few methods. Adding `removeRequest` is the inverse of `addRequest`.

KEY CHANGES

```
removeRequest(request)
    requests.remove(request)  // That's it - just remove from the set
```

3. "What if multiple hall calls come in at the same time?"

Our simulation is single-threaded — you add requests, call step, add more, call step. But in a real system, hall calls arrive concurrently. Two problems show up:

First, two hall calls hitting `requestElevator` at the same time. Both call `selectBestElevator`, both see the same elevator as idle, both dispatch to it. The select + add needs to be atomic.

Second, `step()` is iterating over requests and removing entries while `addRequest` is adding to the same set. That's a concurrent modification.

Two ways to handle this:

Lock around both. Same lock around `requestElevator` and `step()`. Simple and correct — no concurrent access to the set at all. The tradeoff is they block each other: a hall call waits while step is running and vice versa.

LOCK APPROACH

```
requestElevator(floor, type)
    lock.acquire()
    ...
    lock.release()

step()
    lock.acquire()
    ...
    lock.release()
```

Concurrent queue. `addRequest` writes to a thread-safe queue instead of the set directly. A thread safe queue would be like a `BlockingQueue` in Java or a `Queue` in Python. At the start of each tick, `step()` drains the queue into the working set. Writers only touch the queue, step only touches the set. Complete separation — no lock needed, no contention.

CONCURRENT QUEUE APPROACH

```
addRequest(request)
    pendingRequests.enqueue(request)  // thread-safe queue

step()
    while !pendingRequests.isEmpty()
        activeRequests.add(pendingRequests.dequeue())
    // all logic uses activeRequests
    ...
```

The lock approach is simpler to explain. The concurrent queue is more elegant and doesn't block hall calls during step. Either is a good answer.

What is Expected at Each Level?

Here's what interviewers typically expect at each level.

Junior

At the junior level, I want to see that you can model the basic entities and implement straightforward elevator movement. You should identify that you need an Elevator class to track position and direction, and some kind of controller to coordinate multiple elevators. The dispatch logic can be simple—nearest elevator is fine. For movement, I expect basic up/down behavior, and you might need hints to get the "continue in one direction, then reverse when clear" pattern right. It's okay if your elevator bounces back and forth inefficiently at first; the important thing is that it eventually services all requested floors. Edge cases like invalid floor numbers should be handled. If you can demonstrate a working simulation where I can request elevators, add destination floors, and step through time to see elevators move and stop correctly, you've met my expectations. Don't worry about optimizing the dispatch algorithm or handling complex concurrent scenarios.

Mid-level

For mid-level candidates, I expect the "keep going in one direction until no more requests, then reverse" behavior to be implemented correctly with minimal hints. The state machine should be clean: IDLE when no requests, UP or DOWN when moving, proper transitions between states. Your `step()` method should handle the subtle cases—stopping at a floor means you don't move that tick, checking for reversal happens after removing a request, going IDLE when requests are empty. I'm also looking for reasonable dispatch logic. You don't need the priority tier approach, but you should recognize that the naive nearest-elevator strategy has problems and be able to discuss improvements. Your code should be organized so that ElevatorController handles coordination and Elevator handles its own movement. If I ask about extensibility, you should be able to discuss where changes would live—adding hall call direction handling means changing the requests data structure, not rewriting the movement algorithm.

Senior

Senior candidates should produce a design that demonstrates real systems thinking. You should ask the clarifying question about simulation vs. hardware control upfront—that signals you understand the distinction. The entity design should be tight: ElevatorController as stateless coordinator, Elevator with well-encapsulated movement logic. Your movement logic (continue in one direction, reverse when no more requests) should handle edge cases without me pointing them out: boundary floors, stopping and reversing on the same floor, going idle when empty. For dispatch, I want to see the priority tier approach or something equally sophisticated, with clear reasoning about why penalty scores are hacky. You should proactively mention the limitation of not checking whether existing requests will take the elevator past the requested floor. Strong senior candidates discuss tradeoffs fluently: immediate dispatch vs. request queuing, storing floor-only requests vs. floor-direction pairs, simple selection vs. predictive algorithms. If time permits, you should be able to sketch how the design would handle express elevators or priority floors without fundamentally restructuring the existing classes.



Test Your Knowledge

Answer the question below to find your gaps.

Question 1 of 15

Why does the Request class include a RequestType (PICKUP_UP, PICKUP_DOWN, DESTINATION) instead of just storing a floor number?

1. Floor numbers can be negative, which breaks unsigned integer comparisons
2. Request types improve memory efficiency
3. It enables direction-aware stopping decisions
4. Floor numbers alone can't be stored in a Set